

Python Bindings for rencfs

Objectives

- Provide Python developers the ability to access and use the encrypted file system capabilities natively.
- Expose all public methods of `EncryptedFs` to Python, allowing developers to use it similarly to the FUSE implementation.
- Ensure the Python bindings perform with minimal overhead compared to native Rust execution.
- Maintain the same strict security guarantees (like secure memory handling for passwords and zeroization) as the Rust implementation.
- Establish a comprehensive test suite for the Python bindings that mirrors the core functionality testing.
- Integrate the building, testing, and linting of Python bindings into the existing GitHub Actions CI/CD pipeline.

High level

- Set up the environment and select the appropriate tooling for building Rust-Python bindings (focusing on PyO3).
- Map Rust types, structs, and enumerations to Python classes and methods, handling asynchronous functions appropriately.
- Implement the bindings iteratively, starting with basic file system operations (create, open, read, write) and moving to administrative operations (password change).
- Benchmark the bindings against the native Rust implementation to identify and fix any performance bottlenecks at the FFI boundary.
- Document the setup, usage, and provide clear examples for developers integrating the Python library.

Detailed Steps

The setup that will be used throughout this project will be a standard development environment (Linux/Ubuntu 24.04), utilizing Python virtual environments and Cargo.

1. Environment Setup & Tooling Selection

a. Fork the `rencfs` repository on the main branch. b. Clone the repository locally. c. Investigate and validate **PyO3** as the primary tool for bindings (referencing `rencrypt-python` and `zeroize-python`). Compare briefly with SWIG (referencing issue #202) but proceed with PyO3 for its tighter and more ergonomic integration with Rust. d. Set up a Python virtual environment (`python3 -m venv .venv`). e. Install `maturin`, the build system for PyO3 projects (`pip install maturin`). f. Configure `Cargo.toml` or create a new crate (e.g., `rencfs-python`) to support the Python extension library (adding `pyo3` dependency, setting `crate-type = ["cdylib"]`).

2. Define and Export Data Structures

In this part, I will analyze the Rust `EncryptedFs` implementation and expose necessary data

structures to Python. a. Identify all public data structures and enums used by `EncryptedFs` (e.g., `CreateFileAttr`, `FileType`, `Cipher`, `FsError`). b. Create PyO3 wrapper classes and enums (`#[pyclass]`, `#[pyfunction]`) for these structures. c. Implement conversion traits (`FromPyObject`, `IntoPy`) where direct wrapping is not feasible. d. Handle sensitive data (`SecretString`, `SecretVec`) carefully across the FFI boundary to ensure passwords and keys are still zeroized in memory when garbage-collected in Python.

3. Implement Bindings for `EncryptedFs` Core Methods

a. Create a `#[pyclass]` for the main `EncryptedFs` struct. b. Implement the `__new__` method to initialize the filesystem using Python arguments (data directory path, password provider, cipher configuration). c. **Asynchronous Handling:** Since `EncryptedFs` methods in Rust are `async` (using `tokio`), evaluate and implement the best approach for Python. Use `pyo3-asyncio` or wrap them in a blocking runtime (`tokio::Runtime::block_on`) to expose synchronous Python methods, depending on the desired Python API design. d. Implement methods iteratively, mapping Rust functions to Python methods: i. Basic operations: `create`, `open`, `read`, `write`, `flush`, `release`. ii. Directory operations: `read_dir`, `read_dir_plus`. iii. Metadata and structural operations: `rename`, `remove_file`, `remove_dir`, `set_len`. iv. Administrative: `passwd`. e. Write unit tests in Python using `pytest` for each implemented method to ensure they behave exactly like their Rust counterparts.

4. Performance Profiling and Optimization

a. Create a benchmarking script in Python (e.g., using `pytest-benchmark` or custom scripts) to measure read/write throughput for various block sizes (e.g., 4KB, 1MB). b. Run the Python benchmarks and compare them with the native Rust `cargo bench` results (e.g., `bench_writer_1mb_aes256gcm_file`). c. If the overhead introduced by the bindings is significant, use profiling tools (`py-spy` for Python, `perf` for Rust) to identify bottlenecks at the FFI boundary. d. **Zero-copy optimization:** Investigate and implement optimizations for data transfer between Python and Rust. For `read` and `write` operations, use Python's Buffer Protocol (`memoryview`, `PyBuffer` in PyO3) to achieve zero-copy memory access instead of copying byte arrays back and forth.

5. Documentation and CI/CD Integration

a. Write a comprehensive `README-python.md` with installation instructions, build steps, and Python code examples for common workflows. b. Update the existing GitHub Actions workflow (`build_and_tests.yaml`) to include steps for: i. Building the Python wheel using `maturin`. ii. Running the `pytest` suite. c. Create a Pull Request, ask for review, address comments, and merge into the main branch.